



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
з дисципліни «Технології розроблення
програмного забезпечення»
Тема: «Вступ до паттернів проектування»
Варіант «Mind-mapping software»

Виконав:

студент групи ІА-32

Бурдін Д. Є.

Перевірив:

Мягкий М. Ю.

Тема: Вступ до паттернів проектування.

Мета: Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

Тема проєкту: Mind-mapping software

Патерни: Strategy, Prototype, Abstract Factory, Bridge, Composite, SOA

Опис: Візуальний додаток для складання "карт пам'яті" з можливістю роботи з декількома картами (у вкладках), автоматичного промальовування ліній, додавання вкладених файлів, картинок, відеофайлів (попередній перегляд); можливість додавання значків категорій / терміновості, обведення областей карти (поділ пунктирною лінією)

Зміст

Теоретичні відомості.....	2
Хід роботи.....	5
Діаграма класів патерну Strategy	5
Код програми	5
Висновок	9
Питання до лабораторної роботи	10

Теоретичні відомості

Поняття шаблону проєктування

Будь-який патерн проєктування, використовуваний при розробці інформаційних систем, являє собою формалізований опис, який часто зустрічається в завданнях проєктування, вдале рішення даної задачі, а також рекомендації по застосуванню цього рішення в різних ситуаціях. Крім того, патерн проєктування обов'язково має загальновживане найменування. Правильно сформульований патерн проєктування дозволяє, відшукавши одного разу вдале рішення, користуватися ним знову і знову. Варто підкреслити, що важливим початковим етапом при роботі з патернами є адекватне моделювання розглянутої предметної області. Це є необхідним як для отримання належним чином формалізованої постановки задачі, так і для вибору відповідних патернів проєктування.

Відповідне використання патернів проєктування дає розробнику ряд незаперечних переваг. Наведемо деякі з них. Модель системи, побудована в межах патернів проєктування, фактично є структурованим виокремленням тих елементів і зв'язків, які значимі при вирішенні поставленого завдання. Крім цього, модель, побудована з використанням патернів проєктування, більш проста і наочна у вивченні, ніж стандартна модель. Проте, не дивлячись на простоту і наочність, вона дозволяє глибоко і всебічно опрацювати архітектуру розроблюваної системи з використанням спеціальної мови.

Застосування патернів проєктування підвищує стійкість системи до зміни вимог та спрощує неминуче подальше доопрацювання системи. Крім того, важко переоцінити роль використання патернів при інтеграції інформаційних систем організації. Також слід зазначити, що сукупність патернів проєктування, по суті, являє собою єдиний словник проєктування, який, будучи уніфікованим засобом, незамінний для спілкування розробників один одним.

Таким чином шаблони представляють собою, підтверджені роками розробок в різних компаніях і на різних проєктах, «ескізи» архітектурних рішень, які зручно застосовувати у відповідних обставинах.

Шаблон «Strategy»

Призначення: Шаблон «Strategy» (Стратегія) дозволяє змінювати деякий алгоритм поведінки об'єкта іншим алгоритмом, що досягає ту ж мету іншим способом. Прикладом можуть служити алгоритми сортування: кожен алгоритм має власну реалізацію і визначений в окремому класі; вони можуть бути взаємозамінними в об'єкті, який їх використовує. Даний шаблон дуже зручний у випадках, коли існують різні «політики» обробки даних. По суті, він дуже схожий на шаблон «State» (Стан), проте використовується в абсолютно інших цілях – незалежно від стану об'єкта відобразити різні можливі поведінки об'єкта (якими досягаються одні й ті самі або схожі цілі).

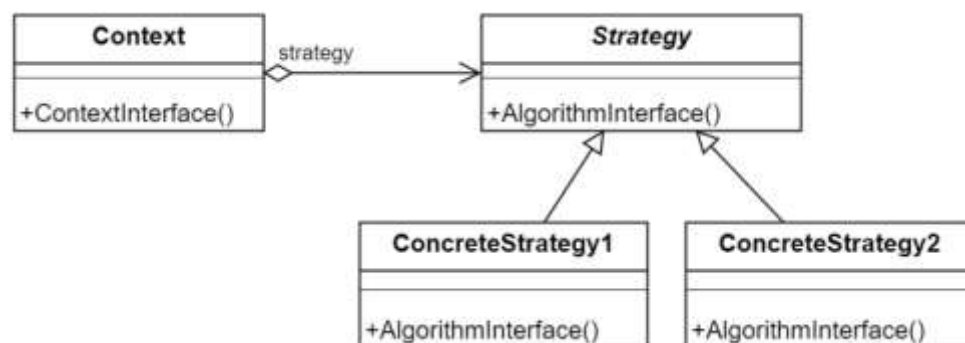


Рис. 1 Структура патерну «Strategy»

Рішення: Коли ви використовуєте патерн «Стратегія», то схожі алгоритми виносяться з класа контекста в конкретні стратегії, за рахунок чого клас контексту стає чистіше і його легше супроводжувати. Також одні і тіж самі стратегії можна використати з різними контекстами, що значно збільшує гнучкість вашої системи та зменшує кількість дублювань у коді.

Контекст при цьому містить посилання на конкретну стратегію, а коли стратегію потрібно замінити, то замінюється об'єкт стратегії в полі `Context.strategy`.

Важливою умовою є відносна простота інтерфейсу для алгоритмів стратегій. Якщо алгоритмам стратегій прийдеться передавати десятки параметрів, то це, скоріш за все, приведе до ускладнення системи та заплутаності коду. Якщо стратегії на вході будуть приймати об'єкт контексту, щоб отримувати з нього всі необхідні дані, то такі стратегії будуть прив'язані до конкретного контексту і їх не можна буде використати з іншим типом контексту.

Приклад з життя: Ви їдете на роботу. Можна доїхати на автомобілі, на метро, або йти пішки. Тут алгоритм, як ви добираетесь на роботу, є стратегією. В залежності від поточної ситуації ви вибираєте стратегію, що найбільше підходить в цій ситуації, наприклад, на дорогах великі пробки тоді ви їдете на метро, або метро тимчасово не ходить тоді ви їдете на таксі, або ви знаходитесь в 5 хвилинах ходьби від місця роботи і простіше добратися пішки.

Переваги та недоліки:

- + Використовувані алгоритми можна змінювати під час виконання.
- + Реалізація алгоритмів відокремлюється від коду, що його використовує.
- + Зменшує кількість умовних операторів типу `switch` та `if` в контексті.
- Надмірна складність, якщо у вас лише кілька невеликих алгоритмів.
- Під час виклику алгоритму, клієнтський код має враховувати різницю між стратегіями.

Хід роботи

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів патерну Strategy

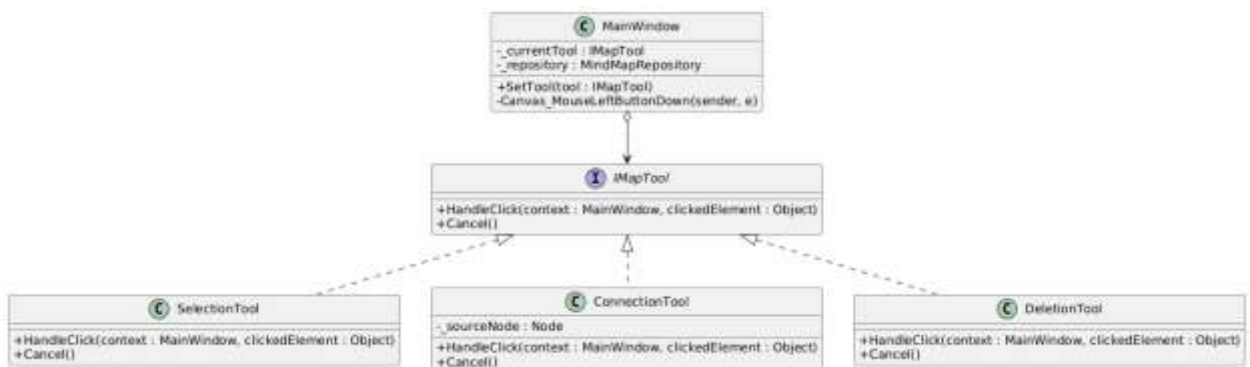


Рис. 2. Діаграма класів патерну Strategy

Код програми

IMapTool (створено)

namespace MindMapApp.Tools

```
{
    public interface IMapTool
    {
        void HandleClick(MainWindow context, object item);
        void Cancel();
    }
}
```

SelectionTool (створено)

```
using MindMapApp.Entities;
using System.Windows.Controls;

namespace MindMapApp.Tools
{
    public class SelectionTool : IMapTool
    {
        public void HandleClick(MainWindow context, object item)
        {
            if (item is Node node)
            {
                var editWindow = new EditNodeWindow(node);
                editWindow.Owner = context;

                if (editWindow.ShowDialog() == true)
                {
                    context.Repository.Update(context.CurrentMap);
                    context.DrawCurrentMap();
                }
            }
        }

        public void Cancel() { }
    }
}
```

ConnectionTool (створено)

```
using System.Windows;
using MindMapApp.Entities;
using System.Linq;
```

```

namespace MindMapApp.Tools
{
    public class ConnectionTool : IMapTool
    {
        private Node _sourceNode;

        public void HandleClick(MainWindow context, object item)
        {
            if (item is Node clickedNode)
            {
                if (_sourceNode == null)
                {
                    _sourceNode = clickedNode;

                    MessageBox.Show($"Початок лінії: {_sourceNode.Text}. Тепер  
оберіть другий вузол.");
                }
                else
                {
                    if (_sourceNode == clickedNode) return;
                    var newConnection = new Connection
                    {
                        FromNodeId = _sourceNode.Id,
                        ToNodeId = clickedNode.Id,
                        MindMapId = context.CurrentMap.Id
                    };

                    if (context.CurrentMap.Connections == null)
                        context.CurrentMap.Connections = new
                            System.Collections.Generic.List<Connection>();
                }
            }
        }
    }
}

```

```

        context.CurrentMap.Connections.Add(newConnection);
        context.Repository.Update(context.CurrentMap);
        _sourceNode = null;
        context.DrawCurrentMap();
    }
}
}
public void Cancel()
{
    _sourceNode = null;
}
}
}

```

DeletionTool (створено)

```

using MindMapApp.Entities;
using System.Linq;
using System.Windows;

```

```

namespace MindMapApp.Tools

```

```

{
    public class DeletionTool : IMapTool
    {
        public void HandleClick(MainWindow context, object item)
        {
            if (item is Node node)
            {
                var result = MessageBox.Show($"Видалити вузол '{node.Text}'?",
                "Видалення", MessageBoxButton.YesNo);
                if (result == MessageBoxResult.Yes)

```



```

    {
        if (context.CurrentMap.Connections != null)
        {
            var linksToRemove = context.CurrentMap.Connections
                .Where(c => c.FromNodeId == node.Id || c.ToNodeId ==
node.Id)
                .ToList();
            foreach (var link in linksToRemove)
            {
                context.CurrentMap.Connections.Remove(link);
            }
        }
        context.CurrentMap.Nodes.Remove(node);
        context.Repository.Update(context.CurrentMap);
        context.DrawCurrentMap();
    }
}

public void Cancel() { }
}
}

```

Висновок: Під час виконання лабораторної роботи було реалізовано патерн «Стратегія» що дозволило інкапсулювати алгоритми взаємодії користувача з робочим полотном (виділення вузлів, створення зв'язків, видалення об'єктів) у незалежні класи-стратегії

Питання до лабораторної роботи

1. Що таке шаблон проектування?
Патерн – це типові рішення для поширеної проблеми, що виникає при проектуванні програмного забезпечення. Це можна назвати описом схеми взаємодії класів та об'єктів, який можна адаптувати до конкретної задачі.
2. Навіщо використовувати шаблони проектування?
Для прискорення процесу розробки завдяки використанню готових рішень, підвищення гнучкості та надійності архітектури, полегшення комунікації при роботі в команді завдяки загальноприйнятим термінам що описують підходи до проектування.
3. Яке призначення шаблону «Стратегія»?
Для визначення сімейства алгоритмів, інкапсуляції кожного з них у власний клас та забезпечення їх взаємозамінності. Це дозволяє обирати або змінювати алгоритм поведінки об'єкта під час виконання програми, не змінюючи сам об'єкт.
4. Які класи входять в шаблон «Стратегія», та яка між ними взаємодія?
Шаблон складається з Контексту (Context), Інтерфейсу Стратегії (Strategy) та Конкретних Стратегій (ConcreteStrategy). Контекст зберігає посилання на об'єкт Стратегії та делегує йому виконання роботи
5. Яке призначення шаблону «Стан»?
Шаблон дозволяє об'єкту змінювати свою поведінку залежно від зміни його внутрішнього стану.
6. Які класи входять в шаблон «Стан», та яка між ними взаємодія?
Контекст (Context), Інтерфейс Стану (State) та Конкретні Стани (ConcreteState). Контекст містить посилання на поточний об'єкт Стану і делегує йому запити. Об'єкти Конкретних Станів реалізують поведінку для певного стану і можуть ініціювати перехід Контексту в інший стан.
7. Яке призначення шаблону «Ітератор»?
Для надання послідовного доступу до елементів складеного об'єкта (колекції) без розкриття його внутрішнього представлення чи структури даних.
8. Які класи входять в шаблон «Ітератор», та яка між ними взаємодія?
Ітератор, Конкретний Ітератор, Агрегат (інтерфейс колекції) та Конкретний Агрегат.
9. В чому полягає ідея шаблону «Одинак»?
Ідея полягає в тому, щоб гарантувати, що певний клас має лише один екземпляр у всій програмі, та надати глобальну точку доступу до цього екземпляра для всіх клієнтів.

10. Чому шаблон «Одинак» вважають «анти-шаблоном»?

Через те, що він створює глобальний стан програми, який важко відслідковувати, порушує принцип єдиної відповідальності (клас контролює і свою логіку, і своє створення) та значно ускладнює модульне тестування (mocking) через жорстку зв'язність компонентів.

11. Яке призначення шаблону «Проксі»?

Надати об'єкт-замінник для іншого об'єкта, щоб контролювати доступ до нього. Це дозволяє перехоплювати виклики до оригінального об'єкта для виконання додаткових дій (перевірка прав доступу, логування).

12. Які класи входять в шаблон «Проксі», та яка між ними взаємодія?

Суб'єкт (Subject), Реальний Суб'єкт (RealSubject) та Проксі (Proxy). Проксі та Реальний Суб'єкт реалізують один інтерфейс. Клієнт звертається до Проксі, який виконує проміжну логіку і за потреби передає виклик Реальному Суб'єкту.